STAT Network Analysis Final Project - UN Voting Patterns

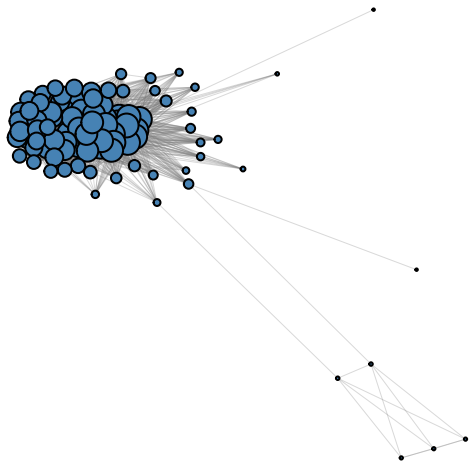
Victory Chukwudi -

2026

```
rm(list=ls())
require(igraph)
load("./data_united_nations.RData")
```

#QUESTION 1

```
# creating a graph from adjacency matrix
gra <- graph_from_adjacency_matrix(A, mode = "undirected")

# adding participation score as vertex attribute
igraph::V(gra)$participation <- participation_score

# removing zero degree nodes for this plot
degrees <- igraph::degree(gra)
gra_plot <- igraph::delete_vertices(gra, which(degrees == 0))

# node sizes proportional to degree
degrees_plot <- igraph::degree(gra_plot)
node_sizes <- 1 + 10 * degrees_plot / max(degrees_plot)

# fixing the layout
set.seed(1)
layout <- layout.fruchterman.reingold(gra_plot)

# plot
plot(gra_plot,
     vertex.size = node_sizes,
     vertex.label = NA,
     vertex.color = "steelblue",
     edge.color = rgb(0.5, 0.5, 0.5, 0.3),
     edge.width = 0.5,
     layout = layout,
     main = "UN Voting Network - Node size by Degree")
```



#QUESTION 2

```
# the various centrality measures
deg <- igraph::degree(gra)
betw <- igraph::centr_betw(gra)$res
pr <- igraph::page_rank(gra)$vector
eigen_cent <- igraph::centr_eigen(gra)$vector
clo <- igraph::centr_clo(gra)$res

# top 3 by each measure
country_names <- igraph::V(gra)$name

cat("Top 3 by Degree:\n")
```

```
## Top 3 by Degree:
```

```
country_names[order(deg, decreasing = T)[1:3]]
```

```
## [1] "PHL" "MEX" "IDN"
```

```
cat("Top 3 by Betweenness:\n")
```

```
## Top 3 by Betweenness:
```

```
country_names[order(betw, decreasing = T)[1:3]]
```

```
## [1] "CYP" "BLR" "PHL"
```

```
cat("Top 3 by Page-Rank:\n")
```

```
## Top 3 by Page-Rank:
```

```
country_names[order(pr, decreasing = T)[1:3]]
```

```
## [1] "PHL" "MEX" "THA"
```

```
cat("Top 3 by Eigenvector:\n")
```

```
## Top 3 by Eigenvector:
```

```
country_names[order(eigen_cent, decreasing = T)[1:3]]
```

```
## [1] "MEX" "LKA" "SEN"
```

```
cat("Top 3 by Closeness:\n")
```

```
## Top 3 by Closeness:
```

```
country_names[order(clo, decreasing = T)[1:3]]
```

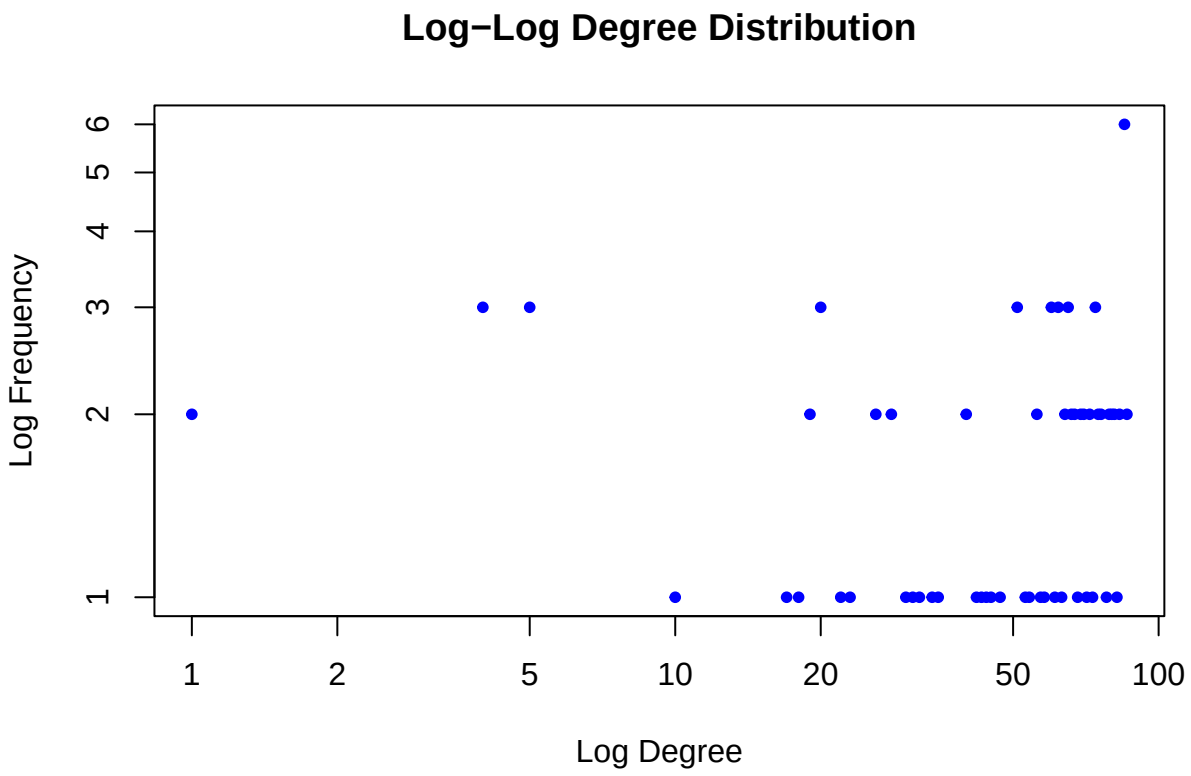
```
## [1] "PHL" "MEX" "IDN"
```

#Question 2 answer

Five centrality measures were computed: degree, betweenness, Page-Rank, eigenvector, and closeness. The three most central countries are **Philippines (PHL)**, **Mexico (MEX)**, and **Indonesia (IDN)**, which consistently rank highly across multiple measures. Cyprus (CYP) and Belarus (BLR) score highest in betweenness, acting as key bridges between voting blocs.

#QUESTION 3

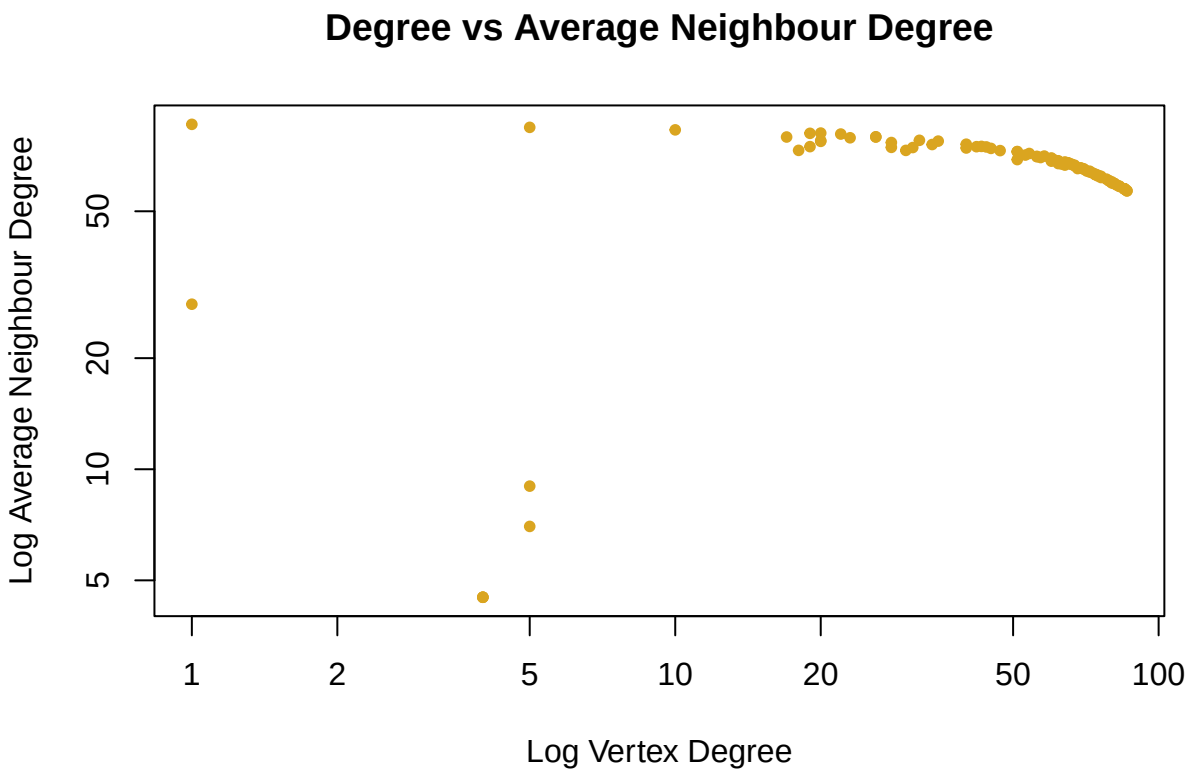
```
# The Power law (log log degree distribution)
deg_freq <- table(deg)
plot(as.numeric(names(deg_freq)), as.numeric(deg_freq),
     log = "xy", pch = 20, col = "blue",
     main = "Log-Log Degree Distribution",
     xlab = "Log Degree", ylab = "Log Frequency")
```




```
# Assortativity
assortativity_degree(gra)
```

```
## [1] -0.2786945
```

```
# Average neighbour degree plot
avg_neigh_deg <- (A %*% deg) / deg
plot(deg, avg_neigh_deg,
     log = "xy", pch = 20, col = "goldenrod",
     main = "Degree vs Average Neighbour Degree",
     xlab = "Log Vertex Degree",
     ylab = "Log Average Neighbour Degree")
```

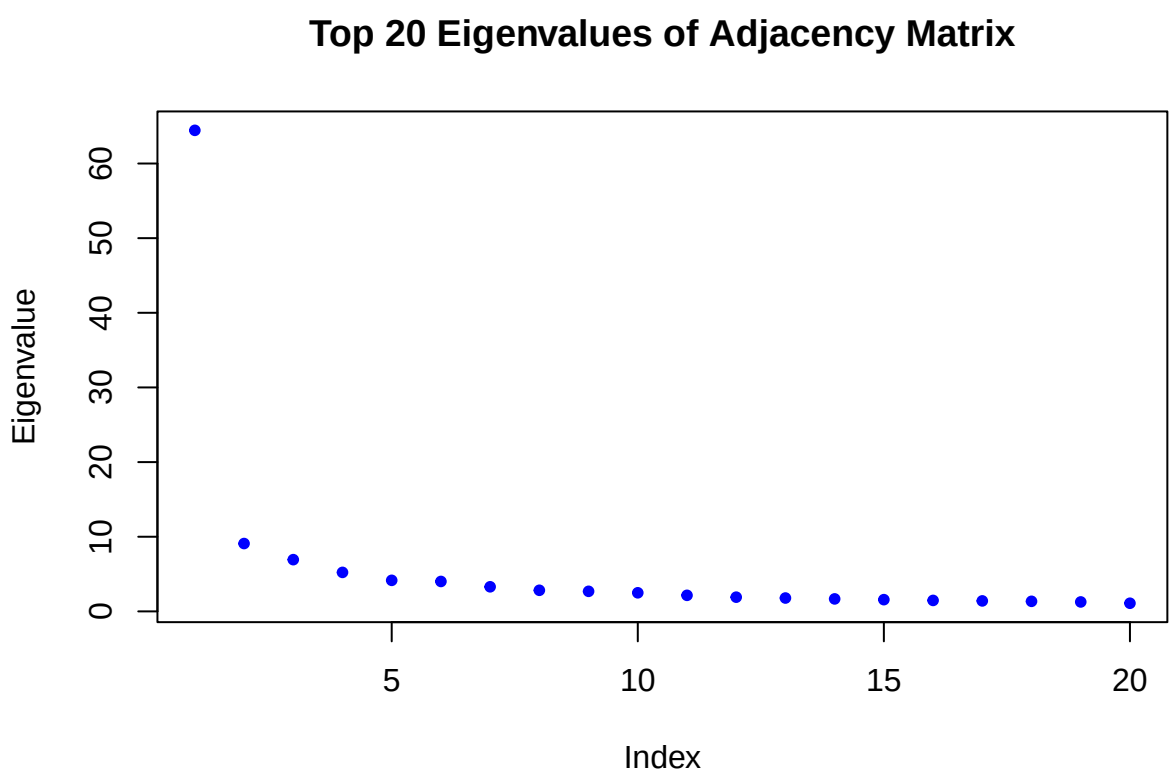


#Question 3 answer

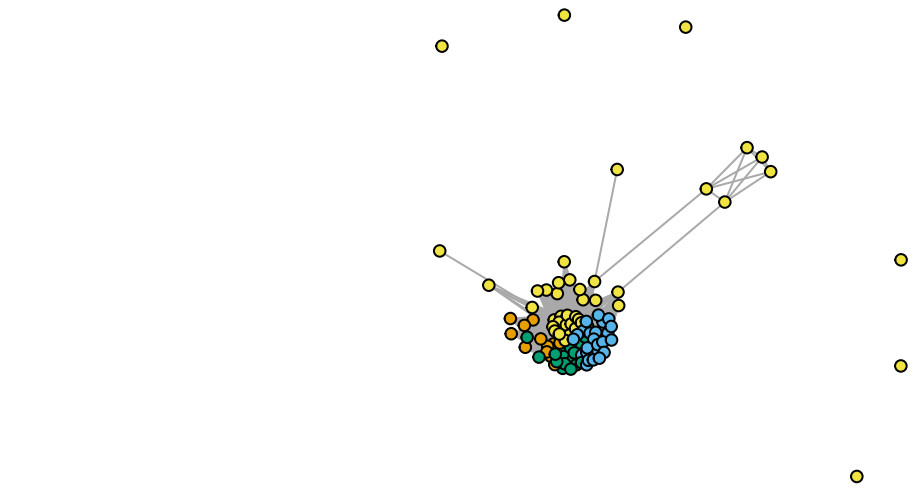
The log-log degree distribution shows no clear straight line, indicating the network does not follow a power law. The assortativity coefficient $r = -0.279$ confirms disassortative mixing highly connected countries tend to connect with less connected ones supported by the average neighbour degree plot, though the effect is weak.

#QUESTION 4

```
# Spectral clustering using adjacency matrix
eigen_decomp <- eigen(A)
n_nodes <- nrow(A)

# plot top 20 eigenvalues
plot(eigen_decomp$values[1:20], col = "blue", pch = 20,
     main = "Top 20 Eigenvalues of Adjacency Matrix",
     xlab = "Index", ylab = "Eigenvalue")
```




```r
# selecting 4 dimensions for clustering
n_dimensions <- 4
selected_dimensions <- 1:n_dimensions
embedding <- eigen_decomp$vectors[, selected_dimensions]

# kmeans clustering with 4 groups
set.seed(1)
memberships <- kmeans(embedding, n_dimensions)$cluster

# plot network with clusters
set.seed(1)
layout <- layout.fruchterman.reingold(gra)
plot(gra,
     vertex.color = memberships,
     vertex.label = NA,
     vertex.size = 5,
     layout = layout,
     main = "Spectral Clustering - 4 Groups")
```




```
# cluster sizes
table(memberships)
```

```
## memberships
##  1  2  3  4
## 12 25 17 46
```

```
# which countries are in each cluster
split(country_names, memberships)
```

```
## $`1`
##  [1] "CHL" "BRA" "COL" "URY" "PAN" "BOL" "ARG" "CRI" "HND" "SLV" "GTM" "DOM"
##
## $`2`
##  [1] "NGA" "GHA" "TGO" "JAM" "BHR" "ZMB" "QAT" "MNG" "TZA" "BFA" "ARE" "SGP"
## [13] "MRT" "KEN" "OMN" "GIN" "MDV" "BRB" "BGD" "MUS" "SLE" "UGA" "BWA" "BTN"
## [25] "NER"
##
## $`3`
##  [1] "PAK" "IRN" "SDN" "SAU" "CUB" "LBY" "DZA" "YEM" "IND" "LBN" "SYR" "NIC"
## [13] "MMR" "VEN" "LAO" "CHN" "AFG"
##
## $`4`
##  [1] "PHL" "MEX" "IDN" "MYS" "THA" "PER" "TUN" "ECU" "EGY" "LKA" "JOR" "SEN"
## [13] "ETH" "MAR" "KWT" "NPL" "GUY" "MLI" "TTO" "BLR" "CYP" "TUR" "POL" "UKR"
```

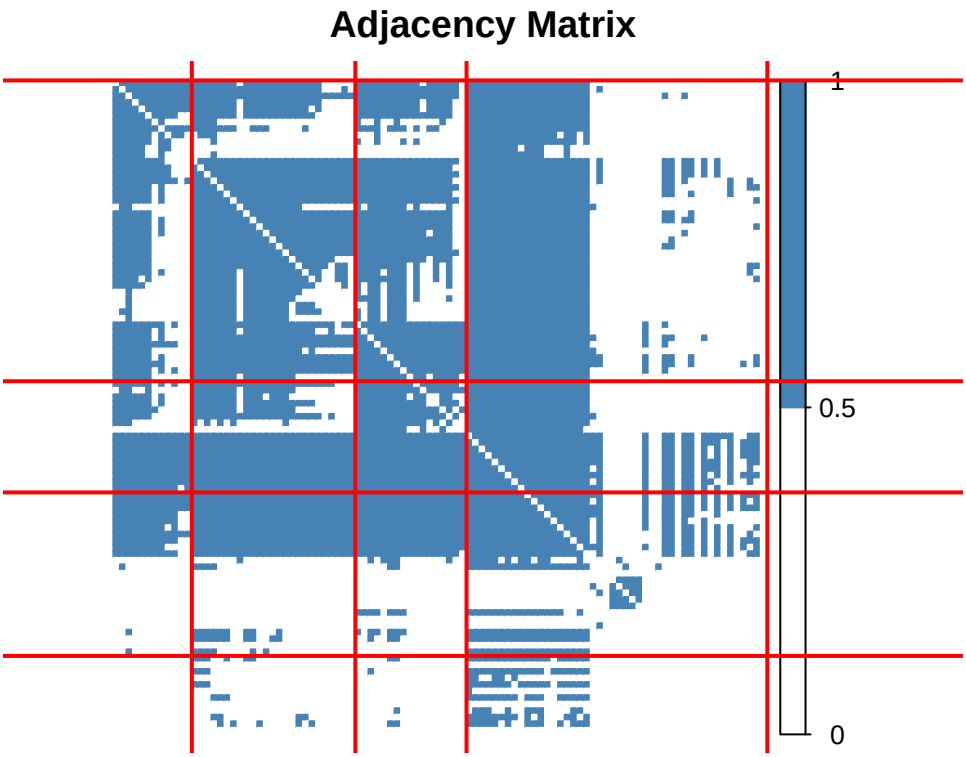
```
## [25] "ROU" "HUN" "BGR" "IRQ" "USA" "MLT" "BEN" "BDI" "GRC" "CIV" "MDG" "NZL"
## [37] "GAB" "HTI" "CMR" "PRY" "LSO" "SWE" "COG" "BHS" "MOZ" "NOR"
```

#Question 4 answer

The eigenvalue plot flattens around index 4–5, supporting $K = 4$. The groups reflect clear geographical blocs: Latin American (Group 1), Sub-Saharan African and Gulf states (Group 2), Middle Eastern and Asian (Group 3), and a large mixed group (Group 4). The structure is clearest for smaller groups, while Group 4 is too mixed to represent a clear bloc.

#QUESTION 5

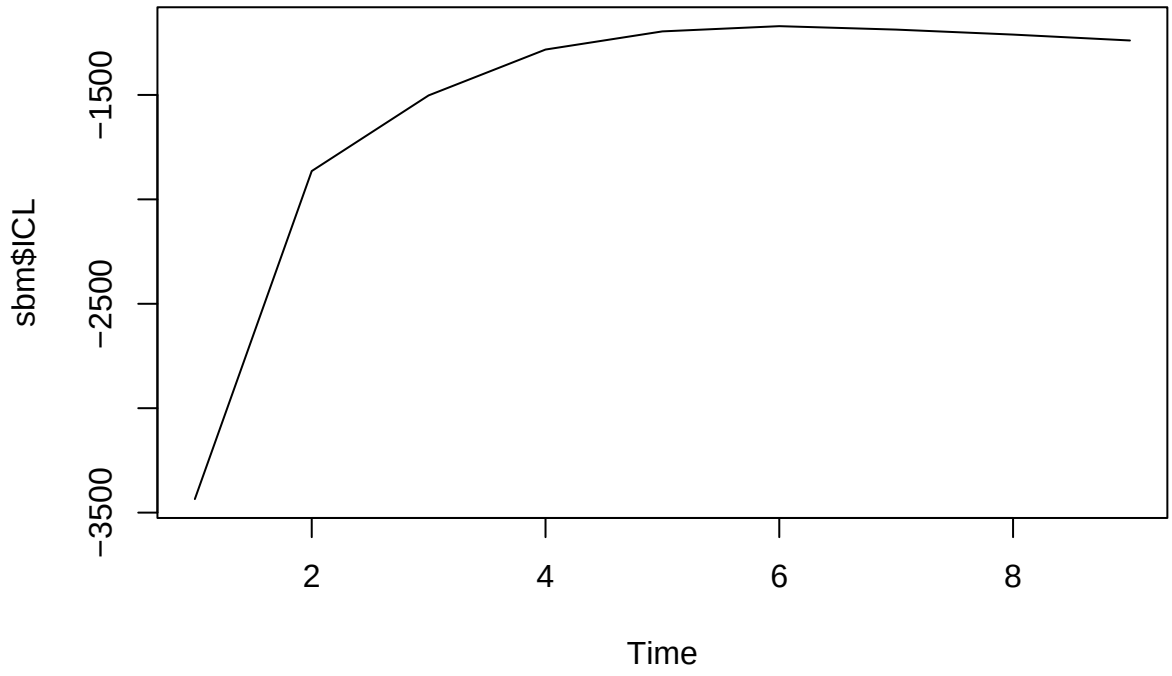
```r
require(corrplot)

# reordering adjacency matrix according to spectral clustering
A_reordered <- A[order(memberships), order(memberships)]

# plot with corrplot
corrplot(A_reordered,
         method = "color",
         is.corr = FALSE,
         tl.pos = "n",
         col = c("white", "steelblue"),
         title = "Adjacency Matrix",
         mar = c(0, 0, 2, 0))

# adding lines to highlight blocks
group_counts <- as.numeric(table(memberships[order(memberships)]))
abline(v = cumsum(group_counts) + 0.5, col = "red", lwd = 2)
abline(h = cumsum(group_counts) + 0.5, col = "red", lwd = 2)
```



#QUESTION 6

```
require(blockmodels)

# fiting SBM
sbm <- BM_bernoulli(membership_type = "SBM_sym",
                    adj = A,
                    verbosity = 0,
                    plotting = "",
                    explore_max = 10)
set.seed(1)
sbm$estimate()
```

```
# choosing best K using ICL
ts.plot(sbm$ICL)
```




```
K_star <- which.max(sbm$ICL)
K_star
```

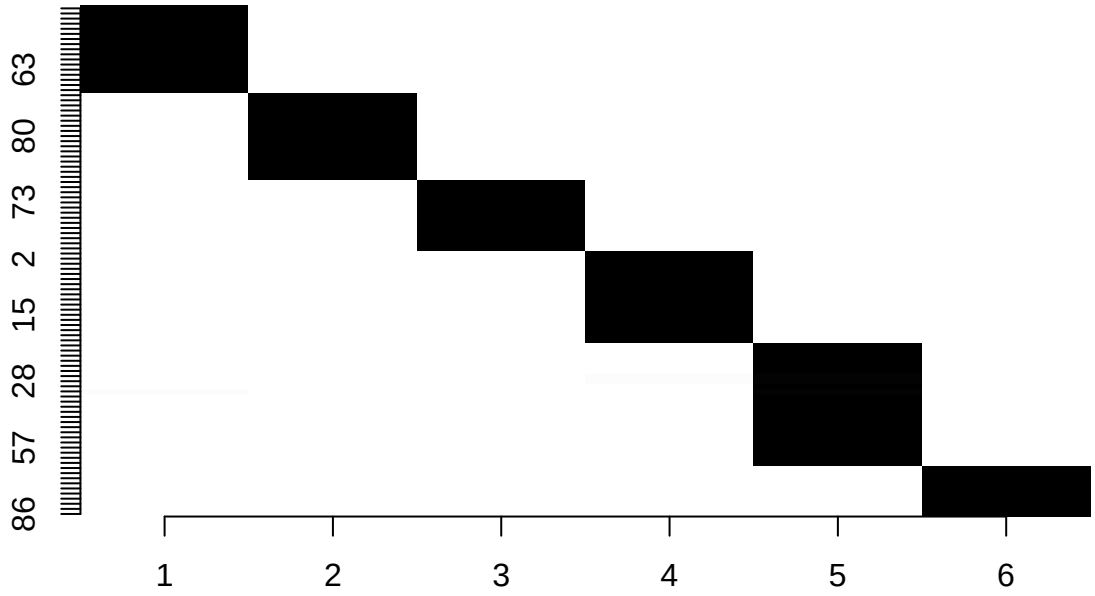
```
## [1] 6
```

```
# extracting clustering
soft_clustering <- sbm$memberships[[K_star]]$Z
hard_clustering <- apply(soft_clustering, 1, which.max)
```

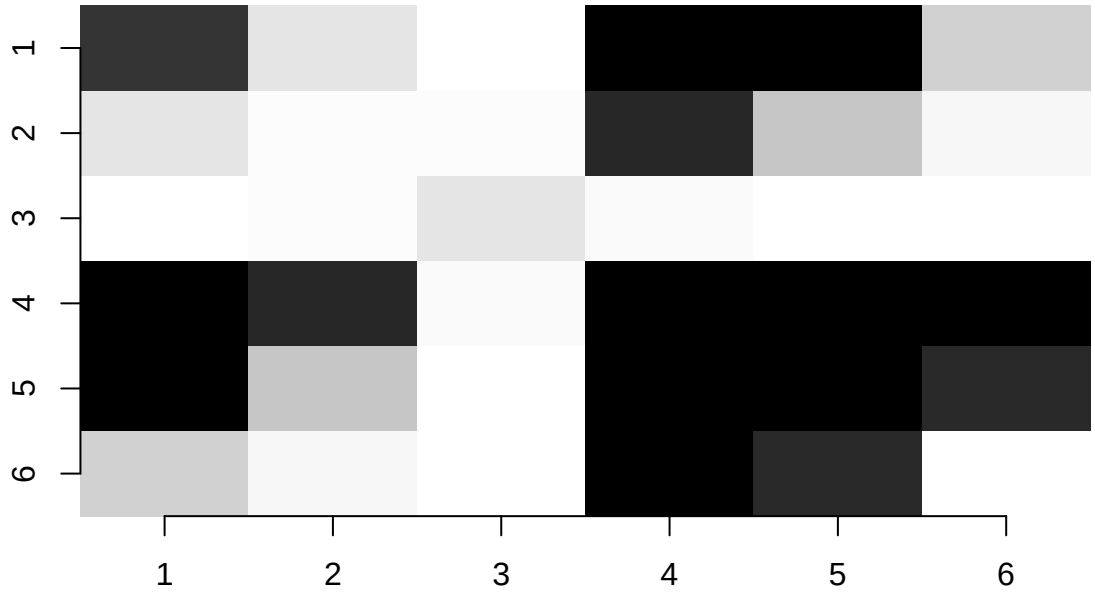
```
# cluster sizes
table(hard_clustering)
```

```
## hard_clustering
##  1  2  3  4  5  6
## 17 17 14 18 24 10
```

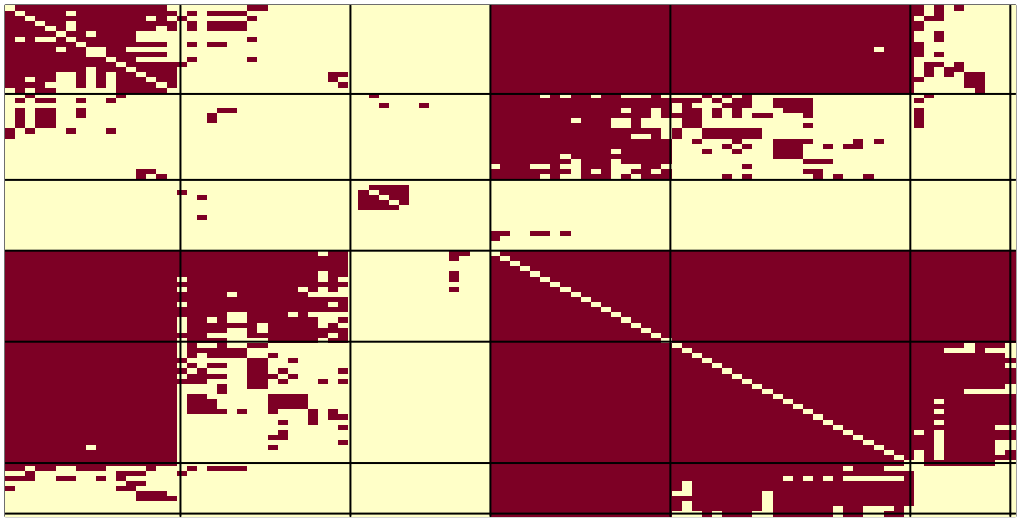
```
# ploting group sizes
sbm$memberships[[K_star]]$plot()
```



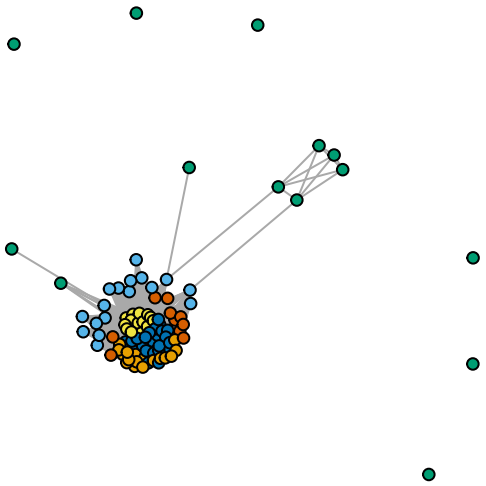

```
# ploting connection probabilities
sbm$plot_parameters(K_star)
```




```r
# ploting reordered adjacency matrix
image(A[order(hard_clustering), rev(order(hard_clustering))],
      xaxt = "n", yaxt = "n",
      main = "SBM - Block Structure")

# highlighting blocks
group_counts <- as.numeric(table(hard_clustering))
abline(v = cumsum(group_counts/sum(group_counts)))
abline(h = 1 - cumsum(group_counts/sum(group_counts)))
```




```
# ploting network with SBM clusters
set.seed(1)
layout <- layout.fruchterman.reingold(gra)
plot(gra,
     vertex.color = hard_clustering,
     vertex.label = NA,
     vertex.size = 5,
     layout = layout,
     main = "SBM Clustering Solution")
```




```
# to know which countries in each group
split(country_names, hard_clustering)
```

```
## $`1`
##  [1] "IRN" "CHL" "CUB" "URY" "PAN" "IND" "SYR" "BOL" "NIC" "MMR" "VEN" "LAO"
## [13] "GIN" "MDV" "BRB" "BGD" "MUS"
##
## $`2`
##  [1] "BLR" "ARG" "CYP" "HND" "SLV" "GTM" "DOM" "AFG" "IRQ" "CIV" "MDG" "GAB"
## [13] "CMR" "LSO" "COG" "BHS" "MOZ"
##
## $`3`
##  [1] "TUR" "POL" "UKR" "ROU" "HUN" "BGR" "USA" "MLT" "GRC" "NZL" "HTI" "PRY"
## [13] "SWE" "NOR"
##
## $`4`
##  [1] "PHL" "MEX" "IDN" "MYS" "THA" "PER" "TUN" "ECU" "EGY" "LKA" "JOR" "SEN"
## [13] "ETH" "KWT" "NPL" "GUY" "MLI" "TTO"
##
## $`5`
##  [1] "PAK" "BRA" "COL" "SDN" "SAU" "LBY" "MAR" "DZA" "YEM" "LBN" "NGA" "GHA"
## [13] "TGO" "JAM" "BHR" "ZMB" "QAT" "TZA" "BFA" "ARE" "SGP" "MRT" "KEN" "OMN"
##
## $`6`
##  [1] "CRI" "MNG" "CHN" "SLE" "UGA" "BWA" "BTN" "NER" "BEN" "BDI"
```

```r
# comparing SBM vs spectral clustering
table(memberships, hard_clustering)
```

```
##            hard_clustering
## memberships  1  2  3  4  5  6
##           1  4  5  0  0  2  1
##           2  5  0  0  0 14  6
##           3  8  1  0  0  7  1
##           4  0 11 14 18  1  2
```

#Question 6 answer

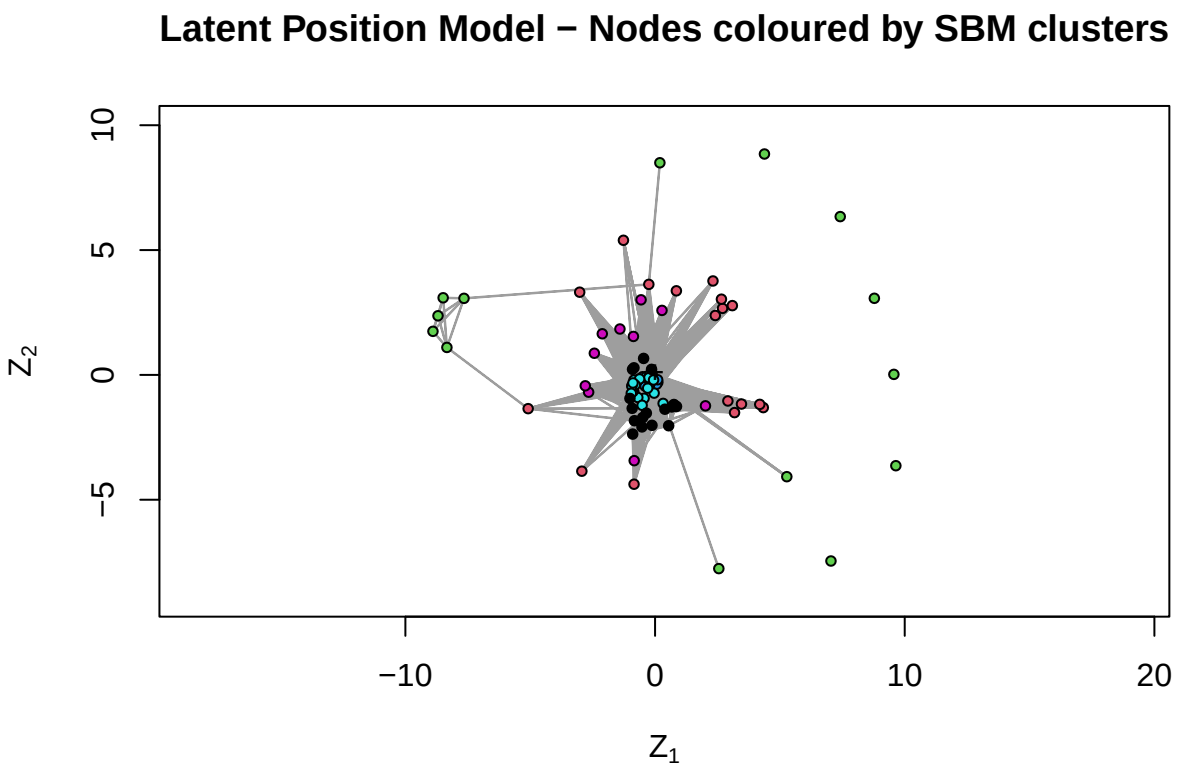
The ICL peaks at $K = 6$ with cluster sizes 17, 17, 14, 18, 24, and 10, confirming a well-defined partition. The groups represent: Latin American and Asian (1), Eastern European and African (2), Western bloc (3), Developing world (4), Islamic and African (5), and a distinct smaller group including China (6). The SBM provides a finer partition than spectral clustering, splitting the large mixed Group 4 across multiple groups, while both methods consistently identify the same broad geographical blocs.

#QUESTION 7

```r
# loading dependencies required for latentnet
library(rlang)
library(tibble)
library(ergm)
library(latentnet)
require(igraph)
load("./data_united_nations.RData")
require(intergraph)

# rebuilding gra and net
gra <- graph_from_adjacency_matrix(A, mode = "undirected")
net <- asNetwork(gra)

# rerunning SBM to get hard_clustering back
require(blockmodels)
sbm <- BM_bernoulli(membership_type = "SBM_sym",
                    adj = A,
                    verbosity = 0,
                    plotting = "",
                    explore_max = 10)
set.seed(1)
sbm$estimate()
```

```r
K_star <- which.max(sbm$ICL)
soft_clustering <- sbm$memberships[[K_star]]$Z
hard_clustering <- apply(soft_clustering, 1, which.max)

# fitting latent position model
set.seed(1)
lpm <- ergmm(net ~ euclidean(d=2), verbose = FALSE)

# plotting with SBM cluster colours
plot(lpm,
     what = "mkl",
```

```
     vertex.col = hard_clustering,
     main = "Latent Position Model - Nodes coloured by SBM clusters",
     print.formula = FALSE,
     labels = FALSE,
     plot.vars = FALSE)
```



#QUESTION 8

```
# extracting parameters from fitted SBM
N <- nrow(A)
pi_mat <- sbm$model_parameters[[K_star]]$pi  # block connection probabilities
lambda <- table(hard_clustering) / N  # mixing proportions

# functioning it to generate random SBM
generate_sbm <- function(N, lambda, pi_mat) {
  # assign nodes to groups based on mixing proportions
  groups <- sample(1:length(lambda), size = N,
                   replace = TRUE, prob = lambda)

  # generating the adjacency matrix
  adj_sim <- matrix(0, N, N)
  for (i in 1:N) {
    for (j in i:N) {
      if (i != j) {
        p_ij <- pi_mat[groups[i], groups[j]]
        adj_sim[i, j] <- rbinom(1, 1, p_ij)
```

```
        adj_sim[j, i] <- adj_sim[i, j]
      }
    }
  }
  return(adj_sim)
}

# the observed statistics from A
gra_obs <- graph_from_adjacency_matrix(A, mode = "undirected")
obs_cc <- transitivity(gra_obs)
obs_density <- edge_density(gra_obs)
obs_apl <- mean_distance(gra_obs, unconnected = TRUE)

cat("Observed clustering coefficient:", obs_cc, "\n")
```

```
## Observed clustering coefficient: 0.8175787
```

```
cat("Observed density:", obs_density, "\n")
```

```
## Observed density: 0.5050505
```

```
cat("Observed average path length:", obs_apl, "\n")
```

```
## Observed average path length: 1.570121
```

```
# simulation study
n_sim <- 100
sim_cc <- c()
sim_density <- c()
sim_apl <- c()

set.seed(1)
for (sim in 1:n_sim) {
  adj_sim <- generate_sbm(N, lambda, pi_mat)
  gra_sim <- graph_from_adjacency_matrix(adj_sim, mode = "undirected")
  sim_cc <- c(sim_cc, transitivity(gra_sim))
  sim_density <- c(sim_density, edge_density(gra_sim))
  sim_apl <- c(sim_apl, mean_distance(gra_sim, unconnected = TRUE))
}

# comparing observed vs simulated
cat("Mean simulated clustering coefficient:", round(mean(sim_cc), 4), "\n")
```

```
## Mean simulated clustering coefficient: 0.81
```

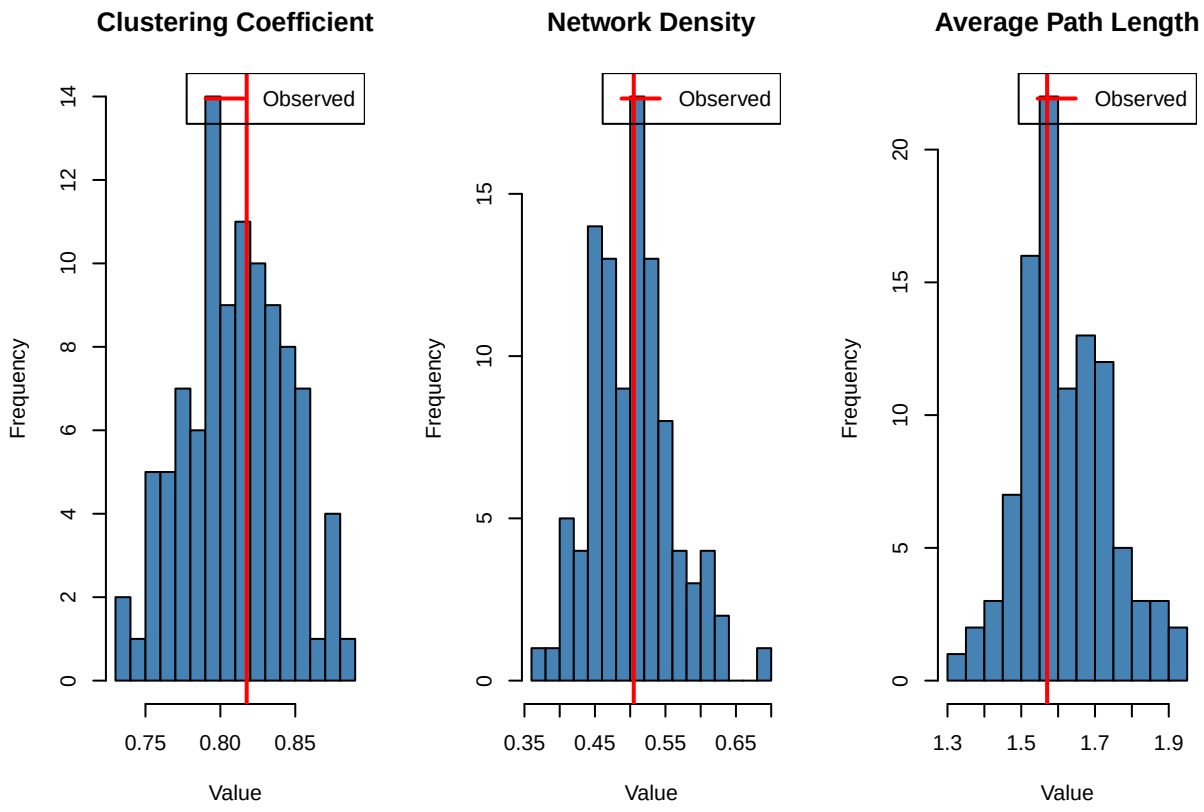
```
cat("Mean simulated density:", round(mean(sim_density), 4), "\n")
```

```
## Mean simulated density: 0.5032
```

```r
cat("Mean simulated average path length:", round(mean(sim_apl), 4), "\n")
```

```
## Mean simulated average path length: 1.6174
```

```r
# ploting comparison
par(mfrow = c(1, 3))

hist(sim_cc, col = "steelblue", main = "Clustering Coefficient",
     xlab = "Value", breaks = 20)
abline(v = obs_cc, col = "red", lwd = 2)
legend("topright", legend = "Observed", col = "red", lwd = 2)

hist(sim_density, col = "steelblue", main = "Network Density",
     xlab = "Value", breaks = 20)
abline(v = obs_density, col = "red", lwd = 2)
legend("topright", legend = "Observed", col = "red", lwd = 2)

hist(sim_apl, col = "steelblue", main = "Average Path Length",
     xlab = "Value", breaks = 20)
abline(v = obs_apl, col = "red", lwd = 2)
legend("topright", legend = "Observed", col = "red", lwd = 2)
```




```r
par(mfrow = c(1, 1))
```

#Question 8 answer

100 random networks were generated from the fitted SBM using λ and π. All observed values fall within the simulated distributions, confirming a good model fit.

#Question 9

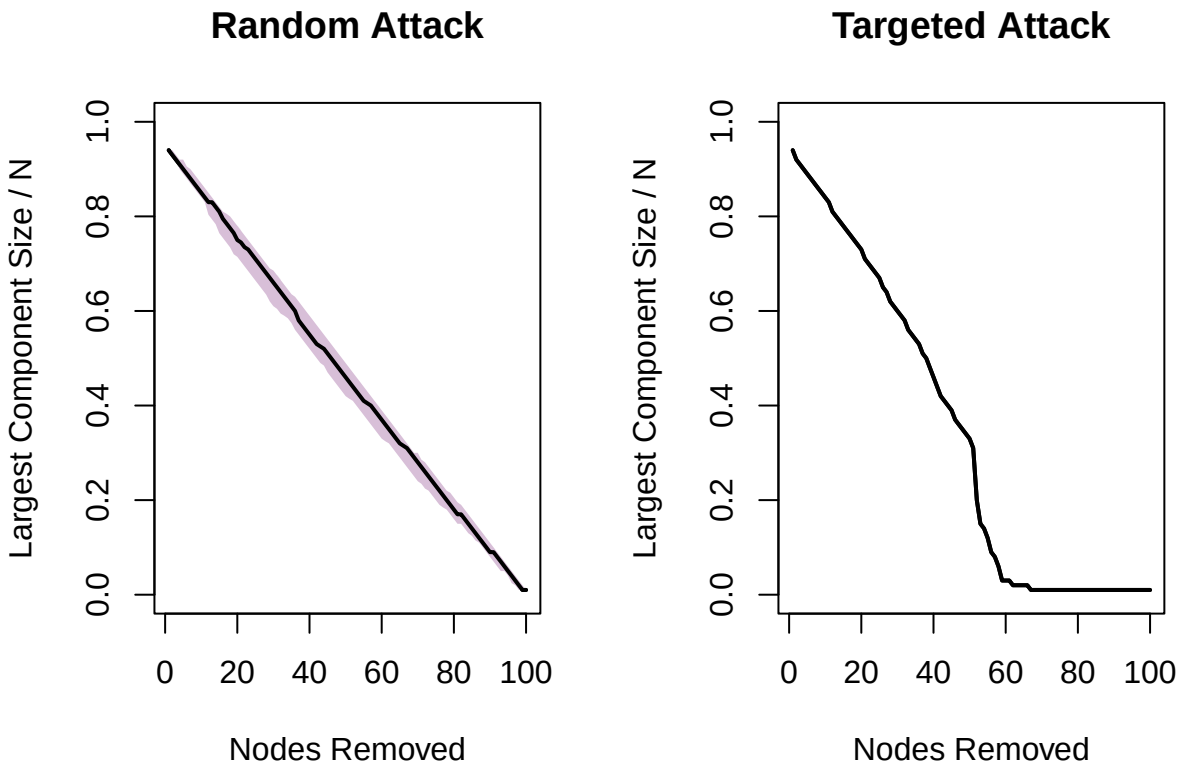
```
# Network Autocorrelation Models
require(sna)
require(numDeriv)

# preparing response variable and explanatory variable
y <- participation_score
intercept <- rep(1, N)
deg <- igraph::degree(gra)

# normalising W to have entries between 0 and 1
W <- A / max(rowSums(A))

# BIC function
bic_formula <- function(res) as.numeric(-2 * res$lnlik.model +
                          log(res$df.total) * res$df.model)

# Model 1: the intercept only + effects model
res1 <- lnam(y = y, x = cbind(intercept), W1 = W, W2 = NULL)
bic1 <- bic_formula(res1)

# Model 2: degree + effects model
res2 <- lnam(y = y, x = cbind(intercept, deg), W1 = W, W2 = NULL)
bic2 <- bic_formula(res2)

# Model 3: intercept only + disturbances model
res3 <- lnam(y = y, x = cbind(intercept), W1 = NULL, W2 = W)
bic3 <- bic_formula(res3)

# Model 4: degree + disturbances model
res4 <- lnam(y = y, x = cbind(intercept, deg), W1 = NULL, W2 = W)
bic4 <- bic_formula(res4)

# rank models by BIC - lower is better
bic_values <- c(bic1, bic2, bic3, bic4)
model_names <- c("Intercept + Effects",
                 "Degree + Effects",
                 "Intercept + Disturbances",
                 "Degree + Disturbances")

results <- data.frame(Model = model_names,
                      BIC = round(bic_values, 4))

# print ranked results
results[order(results$BIC), ]
```

```
##                       Model       BIC
## 4    Degree + Disturbances -376.3942
## 3 Intercept + Disturbances -373.7669
## 1      Intercept + Effects -372.6814
## 2         Degree + Effects -371.1752
```

#QUESTION 10

```r
require(igraph)

# simulation parameters
n_sim <- 50
n_nodes_to_remove <- N - 1

# matrices to store largest component size at each step
store_random <- matrix(NA, n_sim, N)
store_targeted <- matrix(NA, n_sim, N)

# simulate random node removal
set.seed(1)
for (sim in 1:n_sim) {
  gra_temp <- gra
  store_random[sim, 1] <- max(igraph::components(gra_temp)$csize)
  for (i in 1:n_nodes_to_remove) {
    remove_me <- sample(1:length(igraph::V(gra_temp)), 1)
    gra_temp <- igraph::delete_vertices(gra_temp, remove_me)
    store_random[sim, i + 1] <- max(igraph::components(gra_temp)$csize)
  }
}

# simulate targeted node removal by highest degree
set.seed(1)
for (sim in 1:n_sim) {
  gra_temp <- gra
  store_targeted[sim, 1] <- max(igraph::components(gra_temp)$csize)
  for (i in 1:n_nodes_to_remove) {
    remove_me <- which.max(igraph::degree(gra_temp))
    gra_temp <- igraph::delete_vertices(gra_temp, remove_me)
    store_targeted[sim, i + 1] <- max(igraph::components(gra_temp)$csize)
  }
}

# extract median and 90% confidence bounds
data_random <- apply(store_random, 2, quantile,
                     probs = c(0.05, 0.5, 0.95)) / N
data_targeted <- apply(store_targeted, 2, quantile,
                       probs = c(0.05, 0.5, 0.95)) / N

# plot random and targeted attack results side by side
par(mfrow = c(1, 2))

# random attack plot
plot(data_random[2, ], type = "l", lwd = 2,
     main = "Random Attack",
     xlab = "Nodes Removed",
     ylab = "Largest Component Size / N",
     ylim = c(0, 1))
polygon(c(1:N, N:1),
        c(data_random[1, ], rev(data_random[3, ])),
        col = "thistle", border = NA)
```

```r
lines(data_random[2, ], lwd = 2)

# targeted attack plot
plot(data_targeted[2, ], type = "l", lwd = 2,
     main = "Targeted Attack",
     xlab = "Nodes Removed",
     ylab = "Largest Component Size / N",
     ylim = c(0, 1))
polygon(c(1:N, N:1),
        c(data_targeted[1, ], rev(data_targeted[3, ])),
        col = "thistle", border = NA)
lines(data_targeted[2, ], lwd = 2)
```




```r
par(mfrow = c(1, 1))
```

#Question 10 answer

Random removal causes a gradual linear decline in the largest component, while targeted removal of high-degree nodes causes a sharp collapse after ≈ 50 removals. The network is therefore robust to random failures but vulnerable to coordinated attacks, consistent with the disassortative mixing identified in Question 3.